

```
end;
$function$
```

Trigger creation code:

```
CREATE OR REPLACE TRIGGER check_duplicate_user_for_alias_name
BEFORE INSERT OR UPDATE OF name, domain_id ON user_aliases
FOR EACH ROW EXECUTE PROCEDURE check_duplicate_user_for_
alias_name();
```

```
CREATE OR REPLACE TRIGGER check_user_existence BEFORE INSERT
OR UPDATE OF destination, domain_id ON user_aliases FOR EACH
ROW EXECUTE PROCEDURE check_user_existence_for_user_alias();
```

Table: *remote_aliases*

This table is used for storing aliases that are not forwarded to local users. That is, if *foo@example.com* is an alias and it's forwarded to *abc@gmail.com*, then that data is stored here:

```
create table remote_aliases (
id bigserial primary key,
domain_id integer not null references domains(id) on update
restrict on delete cascade,
name character varying(50) not null,
destination character varying(50) not null,
active boolean not null default true,
unique(name, domain_id));
```

Again, a trigger is needed:

```
CREATE OR REPLACE TRIGGER check_duplicate_user_for_alias_name
BEFORE INSERT OR UPDATE OF name, domain_id ON remote_aliases
FOR EACH ROW EXECUTE PROCEDURE check_duplicate_user_for_
alias_name();
```

Table: *quota*

```
create table quota (
username character varying(100) primary key,
bytes bigint not null default 0,
messages integer not null default 0);
```

The trigger for the table is:

```
CREATE OR REPLACE FUNCTION merge_quota()
RETURNS trigger
LANGUAGE plpgsql
AS $function$
BEGIN
IF NEW.messages < 0 OR NEW.messages IS NULL THEN
-- ugly kludge: we came here from this function, really
do try to insert
IF NEW.messages IS NULL THEN
NEW.messages = 0;
```

```
ELSE
NEW.messages = -NEW.messages;
END IF;
return NEW;
END IF;
```

```
LOOP
UPDATE quota SET bytes = bytes + NEW.bytes,
messages = messages + NEW.messages
WHERE username = NEW.username;
IF found THEN
RETURN NULL;
END IF;
BEGIN
IF NEW.messages = 0 THEN
INSERT INTO quota (bytes, messages, username)
VALUES (NEW.bytes, NULL, NEW.username);
ELSE
INSERT INTO quota (bytes, messages, username)
VALUES (NEW.bytes, -NEW.messages, NEW.username);
END IF;
return NULL;
EXCEPTION WHEN unique_violation THEN
-- someone just inserted the record, update it
END;
END LOOP;
END;
$function$
```

```
CREATE OR REPLACE TRIGGER mergequota BEFORE INSERT ON quota
FOR EACH ROW EXECUTE PROCEDURE merge_quota();
```

The above trigger code was taken from the Dovecot wiki <http://wiki2.dovecot.org/Quota/Configuration>

The quota table is used by Dovecot to track the amount of storage used.

Table: *expires*

```
create table expires (
username character varying(100) primary key,
mailbox character varying(255) not null,
expire_stamp integer not null);
```

Trigger for the table -

```
CREATE OR REPLACE FUNCTION merge_expires()
RETURNS trigger
LANGUAGE plpgsql
AS $function$
BEGIN
UPDATE expires SET expire_stamp = NEW.expire_stamp
```